

Mathematical Foundations & Complete Forward Pass of the TabPFN-v3 Architecture

Abstract

This document presents a mathematically comprehensive breakdown of the forward pass of TabPFN-v3 (Tiny configuration). By tracing a concrete batch of the classical Iris dataset ($N = 5$ samples, $F = 3$ features, $C = 3$ classes) through every stage of the model, we illustrate how TabPFN-v3 performs in-context tabular classification purely through a single transformer-based forward pass without test-time parameter updates. Tensor shapes are explicitly tracked across all stages.

1 Problem Setup & Input Dataset

Let the input consist of a single batch ($B = 1$) of a tabular dataset with $N = 5$ samples (consisting of $N_{\text{train}} = 4$ in-context training rows and $N_{\text{test}} = 1$ test row) and $F = 3$ features.

Tensor Definitions & Shapes

- **Input feature tensor (X_{raw}):**

$$X_{\text{raw}} \in \mathbb{R}^{B \times N \times F} = \mathbb{R}^{1 \times 5 \times 3} \quad (\text{Shape: } [1, 5, 3]) \quad (1)$$

- **Target label tensor (y_{raw}):**

$$y_{\text{raw}} \in \mathbb{R}^{B \times N} = \mathbb{R}^{1 \times 5} \quad (\text{Shape: } [1, 5]) \quad (2)$$

The training rows contain their corresponding targets y_i , while the final row ($i = 4$, 0-indexed) is the test row whose label is masked:

$$X_{\text{raw}}[0] = \begin{bmatrix} 5.1 & 1.4 & 0.2 \\ 7.0 & 4.7 & 1.4 \\ 6.3 & 5.1 & 1.8 \\ 5.0 & 1.3 & 0.2 \\ 6.7 & 4.4 & 1.4 \end{bmatrix} \in \mathbb{R}^{5 \times 3} \quad (\text{Shape: } [5, 3]) \quad (3)$$

$$y_{\text{raw}}[0] = [0 \quad 1 \quad 2 \quad 0 \quad 0] \in \mathbb{R}^{1 \times 5} \quad (\text{Shape: } [1, 5]) \quad (4)$$

Where class indices correspond to $\{0 : \text{Setosa}, 1 : \text{Versicolor}, 2 : \text{Virginica}\}$. The final element of $y_{\text{raw}}[0]$ is masked to 0 to prevent target leakage during embedding lookup, and is mathematically neutralized at a later stage.

2 Stage 1: NaN Expansion & Feature Embedding (x_embed)

To capture non-linear relationships directly in the linear embedding layer, each raw scalar feature $x_{i,f}$ is expanded into a 6-dimensional representation vector $\phi(x_{i,f}) \in \mathbb{R}^6$.

Tensor Definitions & Shapes

- **Expanded scalar feature vector** ($\phi(x_{i,f})$): Shape: $[6]$
- **Linear projection weights** (W_{embed}): Shape: $[d, 6] = [48, 6]$
- **Projection bias** ($\mathbf{b}_{\text{embed}}$): Shape: $[d] = [48]$
- **Output projection vector** ($\mathbf{z}_{i,f}$): Shape: $[d] = [48]$
- **Embedded feature tensor** (X_{embed}): Shape: $[B, N, F, d] = [1, 5, 3, 48]$

The expansion vector is constructed using polynomial and logarithmic elements to enrich the scalar's representation space:

$$\phi(x_{i,f}) = \begin{bmatrix} x_{i,f} \\ \mathbb{I}(\text{NaN}) \\ x_{i,f}^2 \\ x_{i,f}^3 \\ \ln(|x_{i,f}| + \epsilon) \\ \text{sign}(x_{i,f}) \end{bmatrix} \in \mathbb{R}^6 \quad (\text{Shape: } [6]) \quad (5)$$

Where $\mathbb{I}(\text{NaN}) \in \{0.0, 1.0\}$ is the missingness indicator, and $\epsilon = 10^{-5}$ ensures numerical stability. For the test row's first feature ($x_{4,1} = 6.7$), the expanded input vector is:

$$\phi(6.7) = \begin{bmatrix} 6.7 \\ 0.0 \\ 44.89 \\ 300.76 \\ 1.902 \\ 1.0 \end{bmatrix} \in \mathbb{R}^6 \quad (\text{Shape: } [6]) \quad (6)$$

Each feature column f is independently projected into the base embedding dimension $d = 48$ using parameters shared across all features:

$$\mathbf{z}_{i,f} = W_{\text{embed}}\phi(x_{i,f}) + \mathbf{b}_{\text{embed}} \in \mathbb{R}^{48} \quad (\text{Shape: } [48]) \quad (7)$$

$$X_{\text{embed}} \in \mathbb{R}^{B \times N \times F \times d} = \mathbb{R}^{1 \times 5 \times 3 \times 48} \quad (\text{Shape: } [1, 5, 3, 48]) \quad (8)$$

—

3 Stage 2: Column-Y Encoding (col_y_encoder)

To encode target relationships, the training label of each row is embedded and injected directly into its feature vectors.

Tensor Definitions & Shapes

- **Label embedding vector** ($\mathbf{e}_c$): Shape: $[d] = [48]$
- **Masked target embedding vector** ($\mathbf{y}_{\text{embed},i}$): Shape: $[d] = [48]$
- **Output label-injected tensor** ($X_{\text{with-y}}$): Shape: $[B, N, F, d] = [1, 5, 3, 48]$

Let the target embedding lookup table be defined as $E_{\text{col}_y} \in \mathbb{R}^{160 \times 48}$ where each active row represents:

$$\mathbf{e}_c = E_{\text{col}_y}[c, :] \in \mathbb{R}^{48} \quad (\text{Shape: } [48]) \quad (9)$$

To prevent leakage, the test row’s label is masked to zeros:

$$\mathbf{y}_{\text{embed},i} = \begin{cases} \mathbf{e}_{y_i} & \text{if } i < 4 \\ \mathbf{0}_{48} & \text{if } i = 4 \end{cases} \quad (\text{Test Row}) \quad (\text{Shape: } [48]) \quad (10)$$

The label embeddings are broadcasted and added element-wise across the F feature dimensions:

$$X_{\text{with}_y}[0, i, f, :] = \mathbf{z}_{i,f} + \mathbf{y}_{\text{embed},i} \in \mathbb{R}^{48} \quad (\text{Shape: } [48]) \quad (11)$$

$$X_{\text{with}_y} \in \mathbb{R}^{1 \times 5 \times 3 \times 48} \quad (\text{Shape: } [1, 5, 3, 48]) \quad (12)$$

—

4 Stage 3: Feature-wise Distribution Encoding

This stage compresses each variable-length row sequence of a feature column into a fixed set of statistical descriptors. We reshape the feature matrix to treat the feature dimension F as the batch dimension.

Tensor Definitions & Shapes

- **Reshaped feature tensor ($X_{\text{per_feature}}$):** Shape: $[B \times F, N, d] = [3, 5, 48]$

$$X_{\text{per_feature}} \in \mathbb{R}^{(B \times F) \times N \times d} = \mathbb{R}^{3 \times 5 \times 48} \quad (\text{Shape: } [3, 5, 48]) \quad (13)$$

—

4.1 Inducing Vectors (Learned Probes)

We introduce a set of $I = 32$ learned global query parameters designed to extract specific distribution metrics.

Tensor Definitions & Shapes

- **Learned inducing matrix (I):** Shape: $[I, d] = [32, 48]$
- **Broadcasted inducing matrix (I_{batch}):** Shape: $[B \times F, I, d] = [3, 32, 48]$

The learned matrix $I \in \mathbb{R}^{32 \times 48}$ is broadcasted along the batch dimension:

$$I \in \mathbb{R}^{32 \times 48} \xrightarrow{\text{broadcast}} I_{\text{batch}} \in \mathbb{R}^{3 \times 32 \times 48} \quad (\text{Shape: } [3, 32, 48]) \quad (14)$$

—

4.2 Cross-Attention Blocks

The inducing queries I_{batch} attend to the row representations $X_{\text{per_feature}}$ across two stacked cross-attention blocks. We detail the projections inside `cross_attn_block1`.

Tensor Definitions & Shapes

- **Projection matrices** (W_Q, W_K, W_V): Shape: $[d, d] = [48, 48]$
- **Query, Key, and Value tensors** (Q, K, V): Shapes: $Q \in [3, 32, 48]$, $K, V \in [3, 5, 48]$
- **Dynamic temperature vector** (τ): Shape: $[B \times F, H_{\text{dist}}, I, 1] = [3, 3, 32, 1]$
- **Head-wise query/key/value** (Q_h, K_h, V_h): Shape: $[B \times F, I, d_{\text{head_dist}}] = [3, 32, 16]$ (for Q_h) and $[3, 5, 16]$ (for K_h, V_h) where $H_{\text{dist}} = 3, d_{\text{head_dist}} = 16$
- **Head-wise attention output** (Attn_h): Shape: $[B \times F, I, d_{\text{head_dist}}] = [3, 32, 16]$
- **Probed output tensor** (X_{probed}): Shape: $[B \times F, I, d] = [3, 32, 48]$

$$Q = I_{\text{batch}} W_Q, \quad K = X_{\text{per.feature}} W_K, \quad V = X_{\text{per.feature}} W_V \quad (15)$$

The dynamic temperature vector τ is computed utilizing a dual MLP structure (global and query-specific):

$$\tau = \text{Softplus}(\text{MLP}_{\text{base}}(1.0) + \text{MLP}_{\text{query}}(Q)) \in \mathbb{R}^{3 \times 3 \times 32 \times 1} \quad (\text{Shape: } [3, 3, 32, 1]) \quad (16)$$

Where $\text{Softplus}(x) = \ln(1 + e^x)$ is applied element-wise. The cross-attention output is calculated per head $h \in \{1, 2, 3\}$:

$$\text{Attn}_h(Q_h, K_h, V_h) = \text{softmax}\left(\frac{Q_h K_h^T}{\tau_h}\right) V_h \in \mathbb{R}^{3 \times 32 \times 16} \quad (\text{Shape: } [3, 32, 16]) \quad (17)$$

The heads are concatenated, projected back through $W_{\text{out_dist}} \in \mathbb{R}^{48 \times 48}$, added to the residual path, and processed by an independent layer normalization and linear sub-block:

$$X_{\text{probed}} \in \mathbb{R}^{3 \times 32 \times 48} \quad (\text{Shape: } [3, 32, 48]) \quad (18)$$

A second, standard cross-attention block (`cross_attn_block2`, using a fixed scaling factor of $1/\sqrt{d_{\text{head_dist}}} = 1/4$) further refines the representation.

4.3 Feature Summary Pooling

The probed representations are mean-pooled across the $I = 32$ inducing points to form a single summary token for each feature column.

Tensor Definitions & Shapes

- **Feature column summary vector** (s_f): Shape: $[d] = [48]$
- **Summary tensor** ($X_{\text{summaries}}$): Shape: $[B, F, d] = [1, 3, 48]$

$$s_f = \frac{1}{32} \sum_{k=1}^{32} X_{\text{probed}}[f, k, :] \in \mathbb{R}^{48} \quad (\text{Shape: } [48]) \quad (19)$$

$$X_{\text{summaries}} \in \mathbb{R}^{B \times F \times d} = \mathbb{R}^{1 \times 3 \times 48} \quad (\text{Shape: } [1, 3, 48]) \quad (20)$$

5 Stage 4: Column Aggregator & Dataset Fingerprint

To synthesize the $F = 3$ isolated column summaries, we prepend two learned summary CLS tokens, $\mathbf{c}_0, \mathbf{c}_1 \in \mathbb{R}^{48}$, to the sequence.

Tensor Definitions & Shapes

- **Learned CLS tokens ($\mathbf{c}_0, \mathbf{c}_1$):** Shape: $[d] = [48]$
- **Prepended token sequence (X_{seq}):** Shape: $[B, F + \text{cls}, d] = [1, 5, 48]$

$$X_{\text{seq}} = [\mathbf{c}_0 \quad \mathbf{c}_1 \quad \mathbf{s}_1 \quad \mathbf{s}_2 \quad \mathbf{s}_3]^T \in \mathbb{R}^{5 \times 48} \quad (\text{Shape: } [5, 48]) \quad (21)$$

5.1 Rotary Position Encoding (RoPE)

We construct rotary frequency matrices $R_{\Theta, m}$ using the frequencies $\theta_k = 100000^{-2k/d_{\text{head_agg}}}$ for $k \in \{0, \dots, 7\}$ (with $d_{\text{head_agg}} = 16$).

Tensor Definitions & Shapes

- **Rotated query and key matrices (Q'_m, K'_m):** Shape: $[B, F + \text{cls}, d] = [1, 5, 48]$
- **Rotary frequency vector (θ):** Shape: $[d_{\text{head_agg}}/2] = [8]$

The RoPE rotation matrix $R_{\Theta, m} \in \mathbb{R}^{16 \times 16}$ for head dimension $d_{\text{head_agg}} = 16$ is block-diagonalized:

$$R_{\Theta, m} = \text{diag}(R_{\theta_1, m}, R_{\theta_2, m}, \dots, R_{\theta_8, m}) \quad (22)$$

Where each block is a 2D rotation matrix:

$$R_{\theta_k, m} = \begin{bmatrix} \cos(m\theta_k) & -\sin(m\theta_k) \\ \sin(m\theta_k) & \cos(m\theta_k) \end{bmatrix} \in \mathbb{R}^{2 \times 2} \quad (23)$$

This is applied head-wise to produce the modified projections:

$$Q'_m = R_{\Theta, m} Q_m, \quad K'_m = R_{\Theta, m} K_m \quad (\text{Shapes: } Q'_m, K'_m \in [1, 5, 48]) \quad (24)$$

Where Q, K are generated by self-attending over X_{seq} .

5.2 Self-Attention and CLS Extraction

We execute self-attention and MLP layers, slice the updated CLS tokens, and concatenate them to expand the dimensions to $d2 = 96$.

Tensor Definitions & Shapes

- **Self-attention output tensor (X_{agg}):** Shape: $[B, F + \text{cls}, d] = [1, 5, 48]$
- **Normalized CLS tokens ($\mathbf{c}'_{0, \text{normed}}, \mathbf{c}'_{1, \text{normed}}$):** Shape: $[d] = [48]$
- **Dataset fingerprint vector ($\mathbf{v}_{\text{dataset}}$):** Shape: $[d2] = [96]$
- **Broadcasted column context tensor ($X_{\text{column_context}}$):** Shape: $[B, N, d2] = [1, 5, 96]$

Let the post-attention normalized CLS vectors be defined as:

$$\mathbf{c}'_{0,\text{normed}} = [0.5, -0.2, 0.8, -0.1]^T \in \mathbb{R}^{48} \quad (\text{Shape: } [48]) \quad (25)$$

$$\mathbf{c}'_{1,\text{normed}} = [-0.3, 0.6, 0.1, 0.4]^T \in \mathbb{R}^{48} \quad (\text{Shape: } [48]) \quad (26)$$

Concatenation yields:

$$\mathbf{v}_{\text{dataset}} = [\mathbf{c}'_{0,\text{normed}} \parallel \mathbf{c}'_{1,\text{normed}}] \in \mathbb{R}^{96} \quad (\text{Shape: } [96]) \quad (27)$$

This is broadcasted to all $N = 5$ rows:

$$X_{\text{column_context}} = \mathbf{1}_{5 \times 1} \otimes \mathbf{v}_{\text{dataset}} \in \mathbb{R}^{1 \times 5 \times 96} \quad (\text{Shape: } [1, 5, 96]) \quad (28)$$

—

6 Stage 5 & 6: Label Embedding and ICL Input Assembly

We embed our targets at the wider dimension $d2 = 96$ using `icl_y_encoder` and sum the context and target embeddings.

Tensor Definitions & Shapes

- **Wider label embedding vector ($\mathbf{y}_{\text{icl},i}$):** Shape: $[d2] = [96]$
- **ICL target tensor (y_{icl}):** Shape: $[B, N, d2] = [1, 5, 96]$
- **Assembled ICL input sequence ($H^{(0)}$):** Shape: $[B, N, d2] = [1, 5, 96]$

Let the wider label encoder matrix be defined as $E_{\text{icl-y}} \in \mathbb{R}^{160 \times 96}$:

$$\mathbf{y}_{\text{icl},i} = \begin{cases} E_{\text{icl-y}}[y_i, :] & \text{if } i < 4 \\ \mathbf{0}_{96} & \text{if } i = 4 \quad (\text{Test Row}) \end{cases} \quad (\text{Shape: } [96]) \quad (29)$$

We construct the input sequence token $\mathbf{h}_i^{(0)}$ by adding the static context to the target vectors:

$$\mathbf{h}_i^{(0)} = X_{\text{column_context}}[0, i, :] + \mathbf{y}_{\text{icl},i} \in \mathbb{R}^{96} \quad (\text{Shape: } [96]) \quad (30)$$

$$H^{(0)} \in \mathbb{R}^{1 \times 5 \times 96} \quad (\text{Shape: } [1, 5, 96]) \quad (31)$$

—

7 Stage 7: In-Context Learning (ICL) Transformer Blocks

We route our sequence $H^{(0)}$ through 2 sequential, identical ICL Transformer layers (`icl_blocks.0` and `icl_blocks.1`). We detail the internals of `icl_blocks.0`.

Tensor Definitions & Shapes

- **Input/Output state tensor ($H^{(0)}$ / $H^{(2)}$):** Shape: $[B, N, d2] = [1, 5, 96]$

—

7.1 Layer Normalization

We normalize each of the 5 row representations independently across the 96 dimensions.

Tensor Definitions & Shapes

- **Normalized representation vector ($\hat{\mathbf{h}}_i$):** Shape: $[d2] = [96]$
- **LayerNorm scale and shift parameters (γ, β):** Shape: $[d2] = [96]$

Let the sample mean μ_i and variance σ_i^2 for row i be:

$$\mu_i = \frac{1}{96} \sum_{k=1}^{96} \mathbf{h}_i^{(0)}[k], \quad \sigma_i^2 = \frac{1}{96} \sum_{k=1}^{96} \left(\mathbf{h}_i^{(0)}[k] - \mu_i \right)^2 \quad (32)$$

The normalization operation is defined as:

$$\hat{\mathbf{h}}_i = \frac{\mathbf{h}_i^{(0)} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \odot \gamma + \beta \in \mathbb{R}^{96} \quad (\text{Shape: } [96]) \quad (33)$$

7.2 Softmax Scaling & Self-Attention

We project $\hat{H}$ using the linear layers $W_Q, W_K, W_V \in \mathbb{R}^{96 \times 96}$.

Tensor Definitions & Shapes

- **Projection matrices (W_Q, W_K, W_V):** Shape: $[d2, d2] = [96, 96]$
- **Query, Key, and Value matrices (Q, K, V):** Shape: $[B, N, d2] = [1, 5, 96]$
- **Head query/key/value (Q_h, K_h, V_h):** Shape: $[B, N, d_{\text{head_icl}}] = [1, 5, 32]$ where $H_{\text{icl}} = 3, d_{\text{head_icl}} = 32$
- **Temperature vector (τ_h):** Shape: $[B, N, 1] = [1, 5, 1]$
- **Attention output (Attn_h):** Shape: $[B, N, d_{\text{head_icl}}] = [1, 5, 32]$
- **Test row attention weights (α_4):** Shape: $[N] = [5]$
- **Post-attention residual vector ($\mathbf{h}_{i,\text{attn}}$):** Shape: $[d2] = [96]$

$$Q = \hat{H}W_Q, \quad K = \hat{H}W_K, \quad V = \hat{H}W_V \quad (34)$$

Inside `icl_attention`, a Softmax Scaling layer calculates the attention temperature parameters dynamically. Let $\mathbf{q}_{i,h} \in \mathbb{R}^{32}$ represent the query vector for row i and head h . We pass the query through a two-branch MLP system (where MLP_{base} handles query-independent base shifts, and $\text{MLP}_{\text{query}}$ processes query-dependent scaling features):

$$\tau_h(\mathbf{q}_{i,h}) = \text{Softplus}(\text{MLP}_{\text{base}}(1.0) + \text{MLP}_{\text{query}}(\mathbf{q}_{i,h})) \in \mathbb{R} \quad (35)$$

The self-attention output is calculated per head $h \in \{1, 2, 3\}$ using the dynamic temperature vector:

$$\text{Attn}_h(Q_h, K_h, V_h)[i] = \sum_{j=1}^5 \left(\frac{\exp\left(\frac{\mathbf{q}_{i,h}^T \mathbf{k}_{j,h}}{\tau_h(\mathbf{q}_{i,h})}\right)}{\sum_{m=1}^5 \exp\left(\frac{\mathbf{q}_{i,h}^T \mathbf{k}_{m,h}}{\tau_h(\mathbf{q}_{i,h})}\right)} \right) \mathbf{v}_{j,h} \in \mathbb{R}^{32} \quad (\text{Shape: } [32]) \quad (36)$$

For the test row query ($i = 4$), the attention weights are:

$$\alpha_4 = \text{softmax}\left(\frac{\mathbf{q}_4 K^T}{\tau}\right) = [0.168 \quad 0.223 \quad 0.244 \quad 0.168 \quad 0.197] \quad (\text{Shape: } [5]) \quad (37)$$

The heads are concatenated, projected back through $W_{\text{out_icl}} \in \mathbb{R}^{96 \times 96}$, and added to the residual path:

$$\mathbf{h}_{i,\text{attn}} = \mathbf{h}_i^{(0)} + W_{\text{out_icl}} \left(\left\| \bigg|_{h=1}^3 \text{Attn}_h(Q_h, K_h, V_h)[i] \right\| \right) \in \mathbb{R}^{96} \quad (\text{Shape: } [96]) \quad (38)$$

7.3 Feed-Forward Network (MLP)

The residual vector is routed through LayerNorm and an MLP.

Tensor Definitions & Shapes

- **Layer 1 weights and bias** ($W_1, \mathbf{b}_1$): Shapes: $W_1 \in [192, 96]$, $\mathbf{b}_1 \in [192]$
- **Layer 2 weights and bias** ($W_2, \mathbf{b}_2$): Shapes: $W_2 \in [96, 192]$, $\mathbf{b}_2 \in [96]$
- **Output row state vector** ($\mathbf{h}_i^{(1)}$): Shape: $[d2] = [96]$
- **Refined state tensor** ($H^{(2)}$): Shape: $[B, N, d2] = [1, 5, 96]$

$$\mathbf{h}_i^{(1)} = \mathbf{h}_{i,\text{attn}} + W_2 \cdot \text{GELU}(W_1 \text{LN}(\mathbf{h}_{i,\text{attn}}) + \mathbf{b}_1) + \mathbf{b}_2 \in \mathbb{R}^{96} \quad (\text{Shape: } [96]) \quad (39)$$

Where the GELU activation is calculated via its precise tanh approximation:

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right) \quad (40)$$

This process is repeated within `icl_blocks.1` to yield:

$$H^{(2)} \in \mathbb{R}^{1 \times 5 \times 96} \quad (\text{Shape: } [1, 5, 96]) \quad (41)$$

8 Stage 8: Output Normalization & Test Slicing

We apply the final output normalization layer `output_norm` and slice the test row.

Tensor Definitions & Shapes

- **Normalized state tensor** ($H_{\text{normed}}^{(2)}$): Shape: $[B, N, d2] = [1, 5, 96]$
- **Isolated test row representation** ($\mathbf{h}_{\text{test}}$): Shape: $[d2] = [96]$

$$H_{\text{normed}}^{(2)} = \text{LayerNorm}(H^{(2)}) \in \mathbb{R}^{1 \times 5 \times 96} \quad (\text{Shape: } [1, 5, 96]) \quad (42)$$

We discard training rows and isolate the test row vector:

$$\mathbf{h}_{\text{test}} = H_{\text{normed}}^{(2)}[0, 4, :] \in \mathbb{R}^{96} \quad (\text{Shape: } [96]) \quad (43)$$

9 Stage 9: Many-Class Decoder

The class prediction is determined using dot-product similarity between the test representation query and each class key.

9.1 Decoder Query Projection

We project our test token $\mathbf{h}_{\text{test}}$ into a 384-dimensional space.

Tensor Definitions & Shapes

- **Projection matrix and bias ($W_Q, \mathbf{b}_Q$):** Shapes: $W_Q \in [384, 96]$, $\mathbf{b}_Q \in [384]$
- **Decoder query vector ($\mathbf{q}_{\text{dec}}$):** Shape: $[H_{\text{dec}} \times d_{\text{head_dec}}] = [384]$
- **Head-wise query vector (Q_h):** Shape: $[d_{\text{head_dec}}] = [64]$ for each head $h \in \{1, \dots, 6\}$

$$\mathbf{q}_{\text{dec}} = W_Q \mathbf{h}_{\text{test}} + \mathbf{b}_Q \in \mathbb{R}^{384} \quad (\text{Shape: } [384]) \quad (44)$$

We split the query vector into the 6 heads:

$$Q_h = \mathbf{q}_{\text{dec}}[(h-1) \times 64 : h \times 64] \in \mathbb{R}^{64} \quad (\text{Shape: } [64]) \quad (45)$$

9.2 Class Key Projections

For each possible class index $c \in \{0, \dots, C-1\}$, we retrieve its embedding and project it.

Tensor Definitions & Shapes

- **Label embedding vector ($\mathbf{e}_{96,c}$):** Shape: $[d2] = [96]$
- **Projection matrix and bias ($W_K, \mathbf{b}_K$):** Shapes: $W_K \in [384, 96]$, $\mathbf{b}_K \in [384]$
- **Projected class key vector ($\mathbf{k}_{\text{dec},c}$):** Shape: $[384]$
- **Head-wise key vector ($K_{h,c}$):** Shape: $[64]$ for each head $h \in \{1, \dots, 6\}$

The keys are projected using Class key embeddings, retrieved from the wider target lookup embedding table $E_{\text{icl}_Y} \in \mathbb{R}^{160 \times 96}$:

$$\mathbf{e}_{96,c} = E_{\text{icl}_Y}[c, :] \in \mathbb{R}^{96} \quad (\text{Shape: } [96]) \quad (46)$$

The key projection is defined as:

$$\mathbf{k}_{\text{dec},c} = W_K \mathbf{e}_{96,c} + \mathbf{b}_K \in \mathbb{R}^{384} \quad (\text{Shape: } [384]) \quad (47)$$

We split the key vectors into the 6 heads:

$$K_{h,c} = \mathbf{k}_{\text{dec},c}[(h-1) \times 64 : h \times 64] \in \mathbb{R}^{64} \quad (\text{Shape: } [64]) \quad (48)$$

9.3 Similarity Logits & Softmax Output

We scale, sum across the 6 heads, and normalize via Softmax.

Tensor Definitions & Shapes

- **Output class logit (logit_c):** Shape: scalar, output tensor shape: $[C] = [3]$
- **Class probability vector (P):** Shape: $[C] = [3]$

The dot product calculation scales the summation by $1/\sqrt{d_{\text{head_dec}}} = 1/8$ to avoid saturated attention bounds:

$$\text{logit}_c = \frac{1}{8} \sum_{h=1}^6 Q_h \cdot K_{h,c} \quad (\text{Shape: scalar}) \quad (49)$$

The logits are converted to final probabilities:

$$P(c) = \frac{\exp(\text{logit}_c)}{\sum_{j=0}^{C-1} \exp(\text{logit}_j)} \quad (\text{Shape: } [3]) \quad (50)$$

Substituting our concrete output values:

$$P = [0.00346 \quad 0.75138 \quad 0.24516] \in \mathbb{R}^3 \quad (\text{Shape: } [3]) \quad (51)$$

The prediction is obtained:

$$\hat{y} = \arg \max_c P(c) = 1 \implies \mathbf{Versicolor} \quad (\text{Shape: scalar}) \quad (52)$$

—

10 Complete Dimension Flow Table

Table 1: Tensor Dimensions and Shapes across the TabPFN-v3 Forward Pass.

Stage	Component / Layer	Tensor Name / Description	Shape
0	Input Data	X_{raw} (Feature matrix)	$[1, 5, 3]$
0	Input Target	y_{raw} (Label vector)	$[1, 5]$
1a	NaN Expansion	Expanded Feature Input	$[1, 5, 3, 6]$
1b	Feature Projection	X_{embed} (Base embedding)	$[1, 5, 3, 48]$
2	Target Lookup	$X_{\text{with_y}}$ (Label-injected features)	$[1, 5, 3, 48]$
3a	Reshaping	$X_{\text{per_feature}}$ (Independent columns)	$[3, 5, 48]$
3c	Cross-Attention Block 1	X_{probed} (Statistical descriptions)	$[3, 32, 48]$
3e	Mean-Pooling	$X_{\text{summaries}}$ (Column descriptions)	$[1, 3, 48]$
4a	Token Prepension	X_{seq} (Pre-attention column sequence)	$[1, 5, 48]$
4d	CLS Extraction & Concat	$\mathbf{v}_{\text{dataset}}$ (Dataset fingerprint)	$[96]$
4d	Broadcasting	$X_{\text{column_context}}$ (Row-wise context)	$[1, 5, 96]$
5	Label Encoding	y_{icl} (Wider label representations)	$[1, 5, 96]$
6	Input Assembly	$H^{(0)}$ (ICL sequence tokens)	$[1, 5, 96]$
7	Transformer Blocks	$H^{(2)}$ (Context-refined row representations)	$[1, 5, 96]$
8	Output Normalization	$H_{\text{normed}}^{(2)}$ (Final layer state)	$[1, 5, 96]$
8	Slicing	$\mathbf{h}_{\text{test}}$ (Isolated test representation)	$[1, 96]$
9a	Query Projection	$\mathbf{q}_{\text{dec}}$ (Decoder query vector)	$[384]$
9b	Key Projection	$\mathbf{k}_{\text{dec}}$ (Class prototype key vectors)	$[3, 384]$
9e	Decoder Softmax	P (Class probability distribution)	$[1, 3]$